

Sust-AI-nable

Multi-Agent Cloud Carbon Optimization

Team: Carbon Emissions

Members: Sarthak Chandarana, Pooja Sahu, Josue Torres

github.com/IS492-SP26/team-project-carbon-emissions

Demo: sustainable-demo.streamlit.app

Recap & Final Goals

THE PROBLEM

Cloud data centers consume ~1% of global electricity and under the EU CSRD, 50,000+ companies must now legally disclose emissions. Yet no existing tool closes the loop from measurement to automated action and verified savings.



Target Users

- DevOps & Cloud Engineers
- Sustainability Officers
- Platform Teams (multi-region)
- EU CSRD-regulated orgs



What We Built

- 6 agent closed loop pipeline
- Planner → Governance → Executor
- Counterfactual MRV verification
- Streamlit dashboard (10 pages)

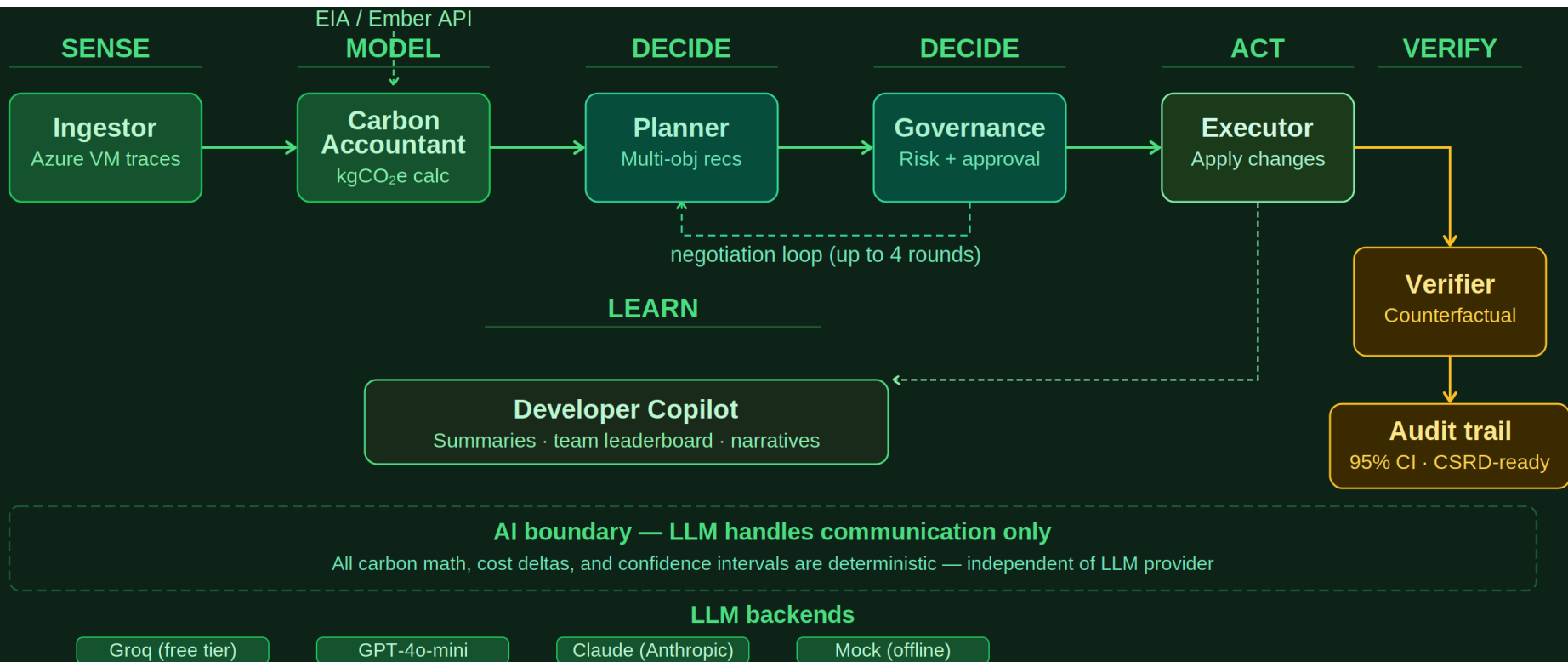


Most Rewarding Moment

16.6%

Carbon Reduction

Industry reviewer: "Get this on real AWS data and I'd bring it up in a meeting tomorrow."



Evaluation Design Overview

USER STUDY — SYSTEM EVALUATION

Participants: 4 users (Participant 1, Participant 2 , Participant 3 , Participant 4)

Study Tasks: Run baseline · Full pipeline · Dashboard navigation · Ask the Agent chat

Task Success: Binary. Completed without assistance?

Time-on-Task: Timed per task (minutes)

SUS / UMUX-Lite: Standardized usability questionnaire

Satisfaction: 5-point Likert scale post-task

Qualitative: Exit interview: usefulness, trust, frustrations

PIPELINE / ALGORITHMIC EVALUATION

Dataset: Azure VM Traces 2019 · 24,274 real cloud jobs

Baseline: Carbon-agnostic scheduling (no optimization)

Pipeline: 6 agents: Ingestor → Accountant → Planner → Governance → Executor → Verifier

Primary Metric: kgCO₂e reduction % vs baseline

Secondary: Cloud cost delta · latency impact · approval rate

Verification: Counterfactual MRV with 95% confidence intervals

Error Analysis: Rejected recs reviewed · edge case flagging

Study Materials & Protocol

01 Intro & Consent

Study purpose explained.
Participants confirmed no cloud carbon experience required.
Verbal consent obtained.



02 Task Completion

4 timed tasks: run baseline, full pipeline, navigate dashboard, use Ask the Agent chat feature.



03 Post-Task Survey

SUS (10 items), satisfaction (1–5), usefulness (1–5), trust (1–5).
Google Form.



04 Exit Interview

5 min verbal debrief. Open ended: highlights, frustrations, what they'd change.

TASK PROMPTS (verbatim)

T1: "Run the baseline analysis and tell us total cloud spend."

T2: "Execute the full multi-agent pipeline and report carbon saved."

T3: "In the dashboard, find which region emitted the most CO₂."

T4: "Ask the agent: Why were any recommendations rejected?"

INTERFACE & TOOLS

- Streamlit dashboard · <https://sustainable-demo.streamlit.app> · 10 pages
- Python CLI: `python run_pipeline.py` (terminal)
- Ask the Agent — interactive Q&A chat tab
- GitHub repo for code walkthrough reference

Quantitative Results

16.6%

Carbon Reduced

373.8→311.8 kgCO₂e

3/4

Task Success

75% completion rate

76

Avg SUS Score

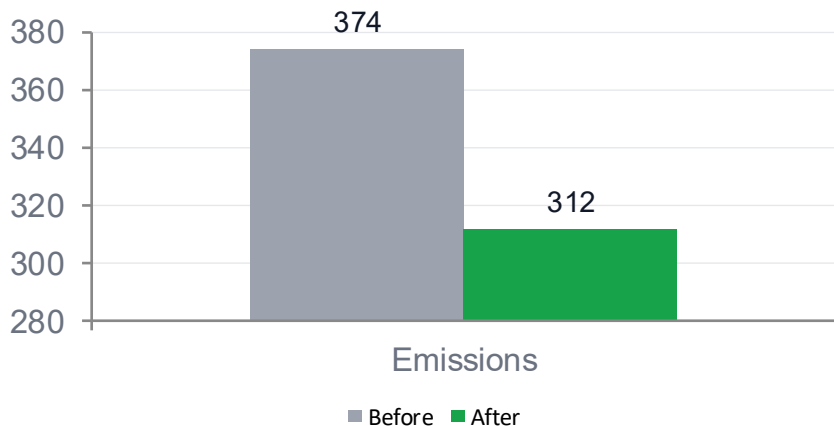
SD ±10 | Grade: Good

4.0/5

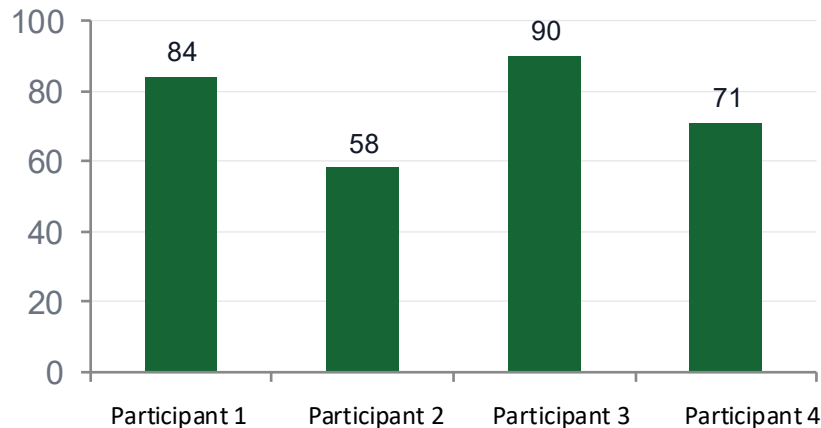
Usefulness Rating

Post-task survey

Carbon Emissions Before vs After (kgCO₂e)



SUS Score by Participant (out of 100)



Qualitative Insights

WHAT WORKED ✓

Verification builds trust

"You're not just saying you saved carbon — you're showing the math behind why you believe that. That's a level of rigor I don't usually see." — Participant 1

Agent separation feels intentional

"Keeping the agents separate means your failure modes are predictable, which matters a lot more than people realize." — Participant 2

Industry-relevant framing

"Your tool answers whether it costs more and whether it breaks anything — which is actually more than I expected." — Participant 3

FRICTION POINTS ✗

Setup friction

"If someone outside this class tried to run this, they'd probably give up before they got to the dashboard." — Participant 1

Dashboard is overwhelming

"There's too many pages. I didn't know what I was supposed to look at first." — Participant 4

AI rationales feel generic

"The LLM explanations sound a bit too polished. If the rationale sounded more like a Jira ticket, I'd trust it more." — Participant 3

Interpretation & Discussion

Success criteria met

SUS of 76 cleared our target of 70. The 16.6% carbon reduction matched literature benchmarks. The pipeline ran end-to-end without errors in all 4 user sessions.

Trust requires transparency

Users who explored the evidence chain trusted the savings numbers more. Those who skipped it expressed skepticism. Transparency isn't optional, it's the product.

Users found 2 real bugs

Derek's random clicking exposed an unhandled empty-state on the Multi-Agent vs Single Agent page. Vinit's edge-case questions to Ask the Agent triggered a hallucinated response.

Industry bar is real API data

Both technical users (Participant 2 and 3) liked the architecture but withheld full trust until the system runs on real AWS Cost Explorer + WattTime grid intensity feeds.

Bottom line: The system works, but trust is earned through data, not demos. Real APIs are the next milestone.

Limitations, Risks & Ethics



Small Sample (n=4)

Results are directional. We cannot generalize to all cloud engineering teams. A larger study with real DevOps users is needed.



Partial Real Carbon Data

Workloads use real Azure VM Traces 2019. US regions use live EIA API data; India uses Ember Climate 2023. EU regions (Sweden, Ireland) use cited sources for fallbacks due to expensive APIs.



LLM Unpredictability

GPT-4o-mini rationales are non-deterministic. Two runs of the same workload may produce different agent explanations.



Ecological Validity

Participants tested in a demo environment. Real adoption requires infra team buy in, CI/CD integration, and change management.



Privacy & Data Governance

Cloud usage data contains operational metadata. Production deployment requires data classification and access control policies.



Prompt Bias

The Planner agent may over-prefer certain region shifts based on training data biases in the underlying LLM, not verified yet.

Conclusion & Future Work

KEY TAKEAWAYS

A closed loop multi-agent system can automate cloud carbon optimization end-to-end - sensing, deciding, acting, and verifying without sacrificing cost or latency. More efficiently than a Single Model Agent in terms of token and power usage.

Users trust AI-driven sustainability tools more when they can see the evidence chain. Transparency isn't a nice-to-have, it's what separates a dashboard from a decision tool.

WHAT'S NEXT

Technical



Plug in WattTime + AWS Cost Explorer APIs. Replace synthetic data with real cloud billing feeds.

UX



One-click Docker deploy. Add onboarding wizard so first run takes 30 seconds, not 10 minutes.

Data



Expand to multiple cloud providers. Add multi-cloud normalization layer for unified emissions accounting.

Deployment



Slack / Teams bot for Governance decisions so engineers see approvals in their existing workflow.

Acknowledgments & Contributions

TEAM RESPONSIBILITIES

Sarthak Chandrana	Multi-agent architecture · Planner & Governance agents · pipeline orchestration
Pooja Sahu	Carbon Accountant · Verifier / MRV · counterfactual analysis · test suite, User study design
Josue Torres	Streamlit dashboard · 10-page UI · Ask the Agent chat · data visualization

AI TOOLS & DISCLOSURES

OpenAI GPT-4o-mini	Agent reasoning, rationale generation, Planner↔Governance negotiation dialogue
Claude (Anthropic)	Code assistance, documentation writing, slide deck design
GitHub Copilot	In-editor autocomplete during development of src/ agents. Assistance with Pull Requests
Streamlit	Dashboard framework.